

ExplainEat

KI für verständliche und ausgewogene Ernährung

Projektanmeldung zur Schweizer KI Challenge 2026

Eingereicht von **Nik**

Stand: 26. Juli 2026 · Projektstatus: Prototyp

Code-Repository	github.com/nik2012slapshot-lgtm/ExplainEat
Video-Pitch (2 Min)	ExplainEat_Pitch_v4.mp4
Screen Recording der App	ExplainEat_App_Demo.mp4
Dieses Dokument	Kritische Reflexion + System- und KI-Dokumentation

Abgabe-Übersicht

Die vier Bestandteile der Abgabe

Geforderter Bestandteil	Wo er zu finden ist
Ausführbarer Code mit README	Dieses Repository, Startanleitung in README.md
2-minütiger Video-Pitch	ExplainEat_Pitch_v4.mp4
Kritische Reflexion	Teil 1 dieses Dokuments, im Repository als kritische_reflexion.md
Offenlegung von Quellen und Unterstützung	Teil 2, Abschnitt 3, sowie Abschnitt 10 (Quellen)

Ergänzend liegt ein Screen Recording der laufenden App bei (ExplainEat_App_Demo.mp4), damit der Ablauf auch ohne lokale Installation nachvollziehbar ist.

Was ExplainEat macht

ExplainEat erkennt Bestandteile einer Mahlzeit auf einem Foto, lässt unsichere Angaben vom Nutzer bestätigen, ergänzt Nährwerte, bildet daraus einen festen numerischen Feature-Vektor, berechnet mit einem **selbst trainierten PyTorch-Modell** einen personalisierten Score und erklärt das bereits geprüfte Ergebnis in verständlicher Sprache.

Der technische Schwerpunkt liegt bewusst auf dem eigenen Machine-Learning-Prozess — Datengenerierung, Feature-Design, Training, Evaluation und Integration — und nicht auf dem visuellen Design der Oberfläche.

Kennzahlen des eigenen KI-Modells

Alle Werte stammen aus einem reproduzierbaren Lauf auf einem **unabhängigen Testanteil** (4 500 Beispiele), der bis zur Schlussauswertung unberührt blieb.

Messgrösse	Wert	Was sie aussagt
Empfehlungsklasse (Accuracy)	98,4 %	Trefferquote über 4 Klassen — hierauf bezieht sich die oft genannte Zahl „98 %“
Flag-Genauigkeit	97,5 %	4 binäre Hinweise (Protein, Ballaststoffe, Zucker, Kalorien)
Score MAE	7,96 Punkte	Mittlerer Fehler auf der Skala 0–100
Score R ²	0,758	Erklärte Varianz
Toleranzquote ±5 Punkte	37,8 %	Anteil der Scores nahe am Zielwert

Vergleich mit Baselines auf demselben Testanteil:

Verfahren	MAE	R ²
Eigenes neuronales Netz	7,96	0,758
Lineare Regression	10,26	0,599
Konstante Vorhersage (Mittelwert)	16,49	−0,000
Regel-Engine	0,00	1,000

Reproduzierbar mit:

```
python scripts/train_ai_model.py --samples 30000 --epochs 40 --seed 42
python scripts/evaluate_model.py --samples 30000 --seed 42
```

Ehrliche Einordnung dieser Zahlen

Drei Punkte sind für eine faire Beurteilung wichtig:

Die „98 Prozent“ betreffen nur die Empfehlungsklasse. Sie sagen aus, wie zuverlässig das Modell eine von vier Kategorien vorschlägt. Sie sind weder die Genauigkeit des Scores noch eine Aussage über ernährungswissenschaftliche Richtigkeit.

Der Score selbst ist deutlich ungenauer. Bei einem mittleren Fehler von rund 8 Punkten und einer Toleranzquote von 37,8 Prozent innerhalb von ± 5 Punkten ist er eine grobe Einordnung, kein präziser Messwert.

Die Regel-Engine erreicht per Konstruktion einen Fehler von null, weil sie die Zielwerte selbst erzeugt. Das neuronale Netz ist damit ein Surrogatmodell: Es lernt, meine eigenen Regeln nachzubilden. Dass es die linearen und konstanten Baselines klar schlägt, belegt die technische Umsetzung und eine sinnvolle Architekturwahl — es belegt nicht, dass die zugrunde liegenden Regeln fachlich korrekt sind. Der nächste inhaltliche Schritt sind fachlich geprüfte Zielwerte, nicht ein grösseres Netz.

Grenzen des Prototyps

- Keine medizinische Beratung, keine Diagnose, keine Therapieempfehlung
- Mengenschätzung aus dem Foto ist ungenau
- Nährwertdatenbank umfasst rund 40 Grundlebensmittel
- Der Allergie-Filter arbeitet mit Schlüsselwörtern und Kategorien und ersetzt die eigene Kontrolle der Zutatenliste nicht
- Trainingsdaten sind synthetisch und regelbasiert, nicht fachlich validiert

Teil 1 — Kritische Reflexion

Projekt: ExplainEat — erklärbare, personalisierte Ernährungs-KI

Eingereicht von: Nik · Schweizer KI Challenge 2026

1. Ziel des Projekts, Zielerreichung und Vorgehen

Ziel. Eine App, die Ernährung nicht nur misst, sondern auch erklärt und personalisiert. Sie soll Lebensmittel auf einem Foto erkennen, eine Mahlzeit anhand eines individuellen Profils bewerten und passende Rezepte vorschlagen oder selbst zusammenstellen. Zum Profil gehören beispielsweise das Körpergewicht, das persönliche Ziel, die körperliche Aktivität und mögliche Allergien.

Zielerreichung. Das Ziel wurde weitgehend erreicht. Es gibt eine funktionsfähige Web-App mit Flutter und einem Python-/Flask-Backend. Die App verfügt über eine Fotoerkennung, eine personalisierte Bewertung mit verständlicher Erklärung, rund 1'000 Rezepte, KI-basierte Empfehlungen und eine KI-gestützte Rezeptgenerierung mit einer an das Körpergewicht angepassten Einkaufsliste.

Noch nicht erreicht ist eine nötige Genauigkeit, sowie eine präzise Schätzung der Mengen auf einem Foto. Diese Einschränkungen werden in den Abschnitten 3 und 4 genauer beschrieben.

Vorgehen / KI-Technologien. ExplainEat verwendet zwei KI-Bausteine:

(a) Ein eigenes neuronales Netz, das mit PyTorch (Open-Source-Framework für maschinelles Lernen und Deep Learning) entwickelt wurde. Dabei handelt es sich um ein mehrschichtiges Perzeptron. Als Eingaben verwendet das Modell Informationen aus dem persönlichen Profil und die Makronährwerte einer Mahlzeit. Als Ausgaben erzeugt es einen Score zwischen 0 und 100, vier Hinweise zu Protein, Ballaststoffen, Zucker und Kalorien sowie eine Empfehlungsklasse.

(b) Eine Bilderkennung mit einem vortrainierten Open-Vocabulary-Segmentierungsmodell. Dieses ist über einen Roboflow-Workflow eingebunden und kann mehrere Zutaten auf einem Foto erkennen.

Das System ExplainEat besteht aus folgenden Bausteinen:

- **Bildanalyse** — Erkennung und Segmentierung der Mahlzeit mit Roboflow und SAM2.
- **Daten- und Wissensbasis** — Nährwerte, Rezepte, Allergene, Profile und Bewertungsregeln.
- **PyTorch-KI-Modell** — Bildung des Feature Vectors, Training und Berechnung des personalisierten Scores.
- **Erklärung und Empfehlungen** — Verständliche Auswertung, Verbesserungen und Rezeptvorschläge.
- **Orchestrierung und Schnittstellen** — Verbindung aller Komponenten über Flask, REST/JSON und die Flutter-App.

ExplainEat ist recht umfangreich geworden. Deshalb habe ich mir erlaubt, zusätzlich eine Projektdokumentation beizulegen, die detaillierte Erläuterungen zu den Bausteinen und zu der Integration und den Schnittstellen liefert.

2. Eigenleistung, Hilfsmittel und KI-Einsatz

Eigenleistung. Zu meiner Eigenleistung gehören die Entwicklung des Konzepts, die Planung der Nutzerführung und die Gestaltung der Anwendung. Ein besonderer Schwerpunkt lag auf der Auswahl und dem Aufbau des eigenen neuronalen Netzes. Dazu gehörten die Architektur, die Codierung der Merkmale und die Trainingsstrategie.

Zusätzlich entwickelte ich die Bewertungslogik und legte die Zielwerte fest. Darauf basiert die Nährwertdatenbank und die Rezeptdaten. Ausserdem integrierte ich den mehrstufigen Workflow für die Bilderkennung, führte Tests durch und die Fehlersuche/Bereinigung.

Besonders anspruchsvoll war die Verbindung des PyTorch-Modells mit dem Python-/Flask-Backend und dem Flutter-Frontend. Dadurch mussten verschiedene Technologien und Datenformate miteinander verbunden werden.

Genutzte Hilfe. Als Programmierunterstützung verwendete ich das KI-Coding-Tool Claude. Das Tool half dabei, anhand meiner Vorgaben Programmcode zu schreiben, Fehler zu finden und mögliche Lösungen vorzuschlagen.

Die Entscheidungen über die Architektur, die Funktionen und die Bewertungslogik wurden jedoch von mir getroffen. Auch die Ergebnisse wurde von mir akribisch überprüft und angepasst. Der von Claude erstellte Code habe ich nicht ungeprüft übernommen.

Wo KI im Produkt steckt.

(1) Das selbst trainierte neuronale Netz von ExplainEat bewertet die Mahlzeiten. Es erzeugt den Score, die Hinweise zu einzelnen Nährwerten und eine Empfehlungsklasse. Die Ergebnisse werden ausserdem für die Auswahl und Zusammenstellung passender Rezepte verwendet.

(2) Das Segmentierungsmodell übernimmt die Bilderkennung und erkennt mehrere Lebensmittel oder Zutaten auf einem Foto.

Für die zentrale Ernährungsbewertung wurde bewusst keine fertige Sprach-KI verwendet. Stattdessen wurde ein eigenes Modell trainiert, damit dessen Aufbau, Eingaben und Ausgaben besser verstanden und erklärt werden können.

3. Was funktioniert — und was (noch) nicht

Funktioniert: Die Anwendung kann auf einem Foto mehrere einzelne Zutaten erkennen. Anschliessend bewertet sie die Mahlzeit anhand des persönlichen Profils und erklärt verständlich, weshalb die Mahlzeit gut oder weniger gut zum gewählten Ziel passt.

Die App enthält rund 1'000 durchsuchbare Rezepte. Die KI-Empfehlungen unterscheiden sich je nach Profil. Bei einem Profil mit dem Ziel Muskelaufbau werden beispielsweise eher proteinreiche Gerichte vorgeschlagen. Bei einem Profil mit dem Ziel Abnehmen werden eher leichtere Gerichte empfohlen.

Weitere funktionierende Bestandteile sind:

- KI-gestützte Rezeptvorschläge
- eine an das Körpergewicht angepasste Einkaufsliste
- Hinweise auf mögliche Nährstoffdefizite
- ein Allergie-Filter, der ungeeignete Rezepte ausblendet

Funktioniert (noch) nicht gut: Die Mengenschätzung auf einem Foto ist noch ungenau. Sie wird momentan hauptsächlich aus der Grösse des erkannten Bereichs auf dem Bild abgeleitet. Ein grosser Bereich auf einem Foto bedeutet jedoch nicht automatisch, dass das Lebensmittel auch ein hohes Gewicht besitzt.

Die Nährwertdatenbank umfasst momentan nur ungefähr 40 Lebensmittel. Bei unbekannten Zutaten greift das System deshalb auf Standard- oder Ersatzwerte zurück. Dadurch kann die Bewertung ungenau werden.

Die Erklärungstexte werden aus den Ergebnissen des Modells und aus vorbereiteten Textbausteinen zusammengesetzt. Sie werden nicht vollständig frei durch eine Sprach-KI formuliert.

Das eigene neuronale Netz lernt aus synthetischen, regelbasierten Trainingsdaten. Es kann die darin enthaltenen Ernährungsregeln gut wiedergeben, wurde bisher aber noch nicht mit genügend echten Daten oder «medizinisch» geprüften Ergebnissen verglichen.

Auch der Login ist bisher nur sehr einfach umgesetzt und deshalb nicht für eine produktive Anwendung mit echten persönlichen Daten geeignet. (Prototyp)

4. Was die KI kann — und was nicht

Kann: Die KI kann eine Mahlzeit für ein bestimmtes Profil bewerten und dazu einen Score sowie eine Begründung ausgeben. Sie kann mögliche Defizite bei Protein oder Ballaststoffen erkennen und auf hohe Werte bei Zucker oder Kalorien hinweisen.

Ausserdem kann sie eine sinnvolle Lebensmittel- oder Rezeptkategorie empfehlen, aus rund 1'000 Rezepten passende Vorschläge auswählen und neue Rezeptvorschläge zusammenstellen. Dabei können das Körpergewicht, das persönliche Ziel und angegebene Allergien berücksichtigt werden.

Die Bilderkennung kann mehrere Zutaten auf einem Foto erkennen.

Kann nicht: Die KI kann keine medizinisch verlässliche oder therapeutische Beratung leisten. Sie kann Erkrankungen wie Diabetes, Stoffwechselstörungen oder Essstörungen nicht sicher berücksichtigen.

Auch versteckte Allergene können nicht vollständig erkannt werden. Besonders bei verarbeiteten oder unbekannten Produkten ist häufig nicht sichtbar, welche Zutaten tatsächlich enthalten sind.

Die KI kann ausserdem keine exakten Grammangaben aus einem Foto bestimmen, keine beliebigen Fragen in natürlicher Sprache verstehen und die Korrektheit ihrer Ergebnisse nicht garantieren.

ExplainEat ist deshalb ein Hilfsmittel zur Orientierung und zum Lernen. Die Anwendung ist kein Ersatz für eine medizinische oder professionelle Ernährungsberatung.

5. Daten: Zuverlässigkeit, Fairness, Bias

Verwendete Daten.

(a) Synthetische Trainingsdaten mit rund 30'000 künstlich erstellten Kombinationen aus persönlichen Profilen und Mahlzeiten. Die dazugehörigen Bewertungen wurden anhand allgemeiner Ernährungsregeln erzeugt. Dazu gehören beispielsweise der ungefähre Proteinbedarf pro Kilogramm Körpergewicht und eine grobe Berechnung des Kalorienbedarfs.

(b) Eine kleine Nährwertdatenbank mit typischen Durchschnittswerten pro 100 Gramm eines Lebensmittels.

(c) Rund 1'000 Rezepte, die überwiegend programmatisch aus echten und üblichen Zutatenkombinationen zusammengestellt wurden.

Zuverlässigkeit. Für einen Prototyp sind die verwendeten Daten brauchbar. Die Bewertungen beruhen jedoch hauptsächlich auf allgemeinen Empfehlungen und Faustregeln. Sie wurden nicht durch eigene klinische Studien bestätigt.

Auch die gespeicherten Nährwerte sind nur Durchschnittswerte. Die tatsächlichen Werte können sich je nach Produkt, Herkunft, Zubereitungsart und Portionsgrösse unterscheiden.

Fairness / Bias (ehrlich):

Demografischer Bias: Die verwendeten Zielwerte orientieren sich an Durchschnittswerten der Bevölkerung. Unterschiede beim Alter, Geschlecht, bei Schwangerschaft, Erkrankungen oder Leistungssport werden nicht ausreichend berücksichtigt. Für diese Personengruppen können die Empfehlungen deshalb unpassend sein.

Körperbau-Bias: Die Skalierung der Portionen verwendet hauptsächlich das Körpergewicht. Sie unterscheidet nicht genau zwischen Muskelmasse, Körperfett, Knochenmasse und Wasseranteil. Dadurch können Portionen über- oder unterschätzt werden.

Kultureller Bias: Die erkennbaren Lebensmittel, die Rezepte und die Nährwertdatenbank sind hauptsächlich westlich und europäisch geprägt. Gerichte aus anderen Kulturen und Regionen werden möglicherweise schlechter erkannt oder nicht passend bewertet.

Sicherheits-Grenze: Der Allergie-Filter erkennt inzwischen auch einige versteckte Allergene über Kategorien. So kann das System beispielsweise erkennen, dass Pesto Nüsse, Tzatziki Milchprodukte und Mayonnaise Ei enthalten kann.

Der Filter arbeitet jedoch weiterhin hauptsächlich mit Schlüsselwörtern und Kategorien. Er kann nicht jedes verarbeitete oder unbekannte Produkt korrekt beurteilen. Deshalb ist er keine medizinische Garantie und darf die eigene Kontrolle der Zutatenliste nicht ersetzen.

6. Zwei kurze Szenarien

a. Positiv. Lena wiegt 68 Kilogramm und hat das Ziel, Muskeln aufzubauen. Sie fotografiert ihr Mittagessen mit ExplainEat.

Die App erkennt Reis und Gemüse und bewertet die Mahlzeit mit 55 von 100 Punkten. ExplainEat erklärt, dass die Mahlzeit zwar Kohlenhydrate und Gemüse enthält, für das Ziel Muskelaufbau jedoch eine ausreichende Proteinquelle fehlt.

Anschliessend empfiehlt die KI ein proteinreicheres Rezept und erstellt eine Einkaufsliste, deren Mengen an Lenas Körpergewicht angepasst sind.

Lena erhält dadurch nicht nur eine Empfehlung, sondern versteht auch, weshalb die ursprüngliche Mahlzeit nicht optimal zu ihrem Ziel passt. Sie kann dieses Wissen später selbst anwenden.

b. Negativ / Missbrauch. Eine Person mit einem problematischen Essverhalten könnte den Kalorien-Score und die Empfehlungen für leichtere Gerichte verwenden, um extreme Einschränkungen zu rechtfertigen. Die Anwendung könnte dadurch unbeabsichtigt ein ungesundes Verhalten verstärken.

Auch beim Allergie-Filter bleibt ein Restrisiko. Häufige versteckte Allergene werden zwar berücksichtigt, ein unbekanntes oder stark verarbeitetes Produkt ausserhalb der gespeicherten Schlüsselwortliste könnte jedoch falsch eingeordnet werden. Dadurch könnte bei der betroffenen Person eine falsche Sicherheit entstehen.

Die Konsequenz daraus ist, dass die App klare Hinweise benötigt:

- ExplainEat bietet keine medizinische Beratung.
- Allergien müssen immer zusätzlich anhand der Zutatenliste geprüft werden.
- Bei gesundheitlichen Problemen muss eine Fachperson kontaktiert werden.
- Für einen praktischen Einsatz wäre eine professionell geprüfte Allergen-Datenbank notwendig.

Teil 2 — System- und KI-Dokumentation

System- und KI-Dokumentation

Projektdokumentation für die Schweizer KI Challenge 2026

Technischer Schwerpunkt

PyTorch-Modell, Feature-Vektor, Training, Inferenz und End-to-End-Integration

Dokumentationsprinzip: Diese Fassung beschreibt das System aus Sicht einer technischen Prüfung. Sie trennt nachweisbare Eigenleistung, externe Komponenten, aktuelle Funktionen, geplante Erweiterungen und noch offene Belege. Unverifizierte Angaben werden nicht als abgeschlossen dargestellt.

Stand: 26. Juli 2026

Projektstatus: Prototyp

Inhalt

1. Projektkontext, Ziel und Schwerpunkt
2. Abgleich mit den Anforderungen der Schweizer KI Challenge 2026
3. Eigenleistung, externe Komponenten und Unterstützungsnachweis
4. Systemarchitektur und Laufzeitablauf
5. Der eigene KI-Kern: PyTorch-Modell und Feature-Vektor
6. Training, Evaluation und Ausbau des Modells
7. Zwei Use Cases und ihr Nutzen
8. Risiken, Datenschutz und fachliche Grenzen
9. Gesamtbeurteilung und offene Punkte vor der Abgabe

Für die Abgabe entscheidend: Die offizielle Challenge erwartet ausführbaren Code mit README, einen zweiminütigen Video-Pitch und eine kritische Reflexion. Zusätzlich muss Unterstützung durch Personen, Institutionen, bestehende Algorithmen, Datensätze oder Werkzeuge transparent ausgewiesen werden.

1. Projektkontext, Ziel und Schwerpunkt

1.1 Entwicklung für die Schweizer KI Challenge 2026

Ich habe ExplainEat für die Schweizer KI Challenge 2026 konzipiert und als funktionsfähigen Prototyp umgesetzt. Im Mittelpunkt steht nicht das visuelle Design der App, sondern die Frage, wie aus einem Foto, bestätigten Lebensmitteldaten und einem Nutzerprofil eine nachvollziehbare, profilbezogene Bewertung entsteht.

Das praktische Problem ist einfach zu beschreiben: Nährwertangaben sind zwar verfügbar, werden aber häufig isoliert betrachtet. Eine Zahl für Kalorien, Zucker oder Protein erklärt noch nicht, wie gut eine konkrete Mahlzeit zu einem Ziel, einer Portionsgrösse oder zu bestätigten Einschränkungen passt. ExplainEat führt diese Informationen in einem kontrollierten Ablauf zusammen und erklärt das Ergebnis in verständlicher Sprache.

Die Dokumentation richtet sich bewusst an zwei Zielgruppen. Nicht-Expertinnen und Nicht-Experten sollen den Nutzen und die Grenzen verstehen. Technische Prüferinnen und Prüfer sollen erkennen können, welche Komponenten ich selbst entwickelt habe, welche extern sind und welche Aussagen erst durch Code, Tests oder ein Screenrecording belegt werden müssen.

ExplainEat in einem Satz: ExplainEat erkennt Bestandteile einer Mahlzeit, lässt unsichere Angaben bestätigen, ergänzt Nährwerte, bildet einen festen numerischen Feature-Vektor, berechnet mit einem trainierten PyTorch-Modell einen Score und lässt ein Sprachmodell das bereits geprüfte Ergebnis erklären.

1.2 Bewusste Abgrenzung: KI vor Design

- Funktionales Frontend statt Designprojekt: Die Flutter-App dient als Eingabe-, Korrektur- und Demonstrationsoberfläche. Branding, Animationen und eine marktreife User Experience waren nicht der Schwerpunkt.
- Eigener Machine-Learning-Prozess: Der grösste technische Aufwand liegt in Datengenerierung, Modelltraining, Evaluation und Integration des Modells in den Laufzeitprozess.
- End-to-End-Integration: Das Modell ist in einen Ablauf aus Bildanalyse, Nutzerbestätigung, Datenanreicherung, Regelprüfung, Flask-API und Erklärungsschicht eingebunden.
- Prototyp statt Medizinprodukt: ExplainEat ist eine Lern- und Forschungsanwendung. Das System ersetzt keine Ernährungsberatung, Diagnose oder medizinische Behandlung.
- Nachvollziehbarkeit vor Inszenierung: Für die Bewertung sind reproduzierbarer Code, klare Metriken, Quellen, Unterstützungsnachweise und eine ehrliche Darstellung der Grenzen wichtiger als eine perfekte Oberfläche.

1.3 Was ich mit ExplainEat nachweisen möchte

- Ich kann ein reales Problem in strukturierte Eingaben, Merkmale und Zielwerte für maschinelles Lernen übersetzen.
- Ich kann synthetische Trainingsdaten erzeugen, deren Herkunft dokumentieren und ihre Aussagekraft kritisch begrenzen.
- Ich kann mit PyTorch ein eigenes neuronales Netz definieren, trainieren, speichern und während der Laufzeit verwenden.
- Ich kann externe KI-Komponenten sauber abgrenzen: Bildanalyse und Sprachmodell unterstützen den Ablauf, ersetzen aber nicht mein eigenes Bewertungsmodell.
- Ich kann Fehlerfortpflanzung, Unsicherheit, Verzerrungen, Datenschutz und fachliche Grenzen systematisch beschreiben.

1.4 Wichtigste technische Aussage

Die eigentliche technische Leistung besteht aus dem formulierten Lernproblem, dem Datenmodell, dem Feature-Vektor, der Modellarchitektur, dem Trainingsprozess, der Evaluation und der Integration in die Anwendung. Diese Unterscheidung ist für die Bewertung der Eigenleistung zentral.

2. Abgleich mit den Anforderungen der Schweizer KI Challenge 2026

2.1 Verlangte Bestandteile der Projektabgabe

Offizielle Anforderung	Umsetzung für ExplainEat	Prüfstatus
Ausführbarer Code mit kurzer Anleitung / README	Backend, Trainingsskript, Inferenzcode, Modellartefakt, Abhängigkeiten, Konfiguration und Beispielinput müssen als reproduzierbares Paket vorliegen. Falls die Ausführung auf einem fremden Computer nicht möglich ist, sollte ein Screenrecording den Ablauf belegen.	Dokumentiert; technisch zu prüfen. Erfüllt

Offizielle Anforderung	Umsetzung für ExplainEat	Prüfstatus
2-minütiger Video-Pitch	Der Pitch zeigt Motivation, Ziel, verwendete KI-Methode, kurze Demo, Ergebnis, wichtigste Herausforderung und ehrliche Grenze.	Erfüllt
Kritische Reflexion	Zielerreichung, Eigenleistung, Daten, Methoden, Metriken, Fehlerquellen, Unsicherheit, Bias, Missbrauchsrisiken und Grenzen werden schriftlich reflektiert.	Erfüllt
Quellen und Unterstützung offenlegen	Frameworks, externe Modelle, öffentliche Daten, Coding-Unterstützung, Coaching und weitere Hilfen werden mit Art und betroffenem Projektteil aufgeführt.	Erfüllt

2.2 Bewertungskriterien – Prüfmatrix

Kriterium	Nachweis in der Dokumentation	Auditstatus
Eigenständigkeit	Eigener Problemzuschnitt, Bewertungslogik, Datengenerator, Feature-Vektor, PyTorch-Modell, Training und Integration. Externe Komponenten werden getrennt ausgewiesen.	Inhaltlich erfüllt
Funktionalität des Codes	Soll-Ablauf und Schnittstellen sind beschrieben. Der Nachweis erfolgt durch Ausführung auf einem fremden System oder durch ein vollständiges Screenrecording.	Zu prüfen durch Audit
Lesbarkeit des Codes	README, Modulstruktur, Konfiguration, Beispielinput, Fehlerbehandlung und Kommentare an nicht offensichtlichen Stellen.	Am Repository prüfen
Schwierigkeitsgrad und Aufwand	Datengenerierung, Preprocessing, neuronales Netz, REST-Integration, Bildanalyse, Regelwerk und Erklärungsschicht bilden einen anspruchsvollen End-to-End-Prototyp.	Plausibel; durch Code belegen
Originalität und Kreativität	Kombination aus visueller Erfassung, Nutzerkorrektur, profilbezogener ML-Bewertung und erklärender Ausgabe.	Inhaltlich erfüllt
Erkenntnisgewinn und Relevanz	Das Projekt zeigt, wie Datenqualität, Portionsunsicherheit, Profilkontext und Modellgrenzen das Ergebnis beeinflussen.	Inhaltlich erfüllt
Wissenschaftliches Arbeiten	Fehlerquellen, Baseline-Vergleich (linear und konstant), getrennter Testanteil, Metriken und Reproduzierbarkeit sind in Anhang B mit gemessenen Werten belegt.	Erfüllt
Selbstkritische Einschätzung	Synthetische Labels und die 98-Prozent-Aussage werden korrekt eingeordnet. Medizinische Grenzen werden ausdrücklich benannt.	Erfüllt

Zentrale Feststellung: Die grösste fachliche Stärke ist die transparente Trennung der Komponenten. Die grösste offene Frage ist, ob das trainierte Modell gegenüber der direkten Regelberechnung einen messbaren Nutzen liefert. Genau dieser Vergleich sollte in Code und Video gezeigt werden.

3. Eigenleistung, externe Komponenten und Unterstützungsnachweis

3.1 Meine eigene technische Arbeit

- Ich habe Problemstellung, Zielgruppe und fachliche Abgrenzung von ExplainEat definiert.
- Ich habe die Bewertungslogik und die Regeln für unterschiedliche Profile und Mahlzeiten entwickelt.
- Ich habe einen Generator für rund 30'000 synthetische Trainingsbeispiele konzipiert und eingesetzt.
- Ich habe die Eingabemerkmale und die feste Reihenfolge des Feature-Vektors festgelegt. Die finale Reihenfolge muss mit dem Code und dem Preprocessing-Artefakt übereinstimmen.
- Ich habe das neuronale Netz in PyTorch aufgebaut mit Code unterstützung, trainiert, validiert, gespeichert und in die Inferenz des Backends eingebunden.
- Ich habe das Flask-Backend als Orchestrierungsschicht zwischen App, Bildanalyse, Datenebene, Regelwerk, PyTorch-Modell und Erklärungskomponente umgesetzt.
- Ich habe Allergie-, Rezept- und Empfehlungslogik sowie die Zuordnung von Lebensmitteln zu Nährwerten aufgebaut.
- Ich habe eine funktionale Flutter-Oberfläche erstellt, ohne den Schwerpunkt auf visuelles Produktdesign zu legen.

3.2 Verwendete Frameworks, Modelle und Hilfsmittel

Komponente	Rolle im System	Meine Eigenleistung	Externer Anteil
PyTorch	Framework für Tensoren, neuronales Netz, Training und Inferenz.	Lernproblem, Feature-Auswahl, Datengenerator, Modellaufbau, Training, Tests und Integration.	Framework, Autograd, Standard-Layer, Optimizer und Daten-Utilities.
SAM2 / Roboflow	Bildbereiche segmentieren und – je nach eingesetztem Modell – Objekte oder Lebensmittelklassen erkennen.	Anbindung, Datenfluss, Konfidenzbehandlung, Fehlerfälle und Übergabe an die eigene Bewertung.	Vortrainierte Segmentierungsmodelle, Hosting- oder Inferenzfunktionen und gegebenenfalls externe Erkennungsmodelle.
Visual Studio (VS), Claude Code via PlugIN (VS Code)	Genrierung von Code	Promptstruktur, erlaubte Eingaben, Ausgaberegeln, Sicherheitsgrenzen und Systemintegration. Prüfen des Codes, Testing	Sprachmodell und dessen allgemeine Sprachfähigkeit. Claude berechnet nicht den verbindlichen Score.
Flask	REST-Backend und technische Orchestrierung.	API-Endpunkte, Validierung, Workflow, Fehlerbehandlung, Modellaufruf und Ergebnisobjekt.	Webframework und Standardfunktionen.
Flutter	Funktionale Eingabe-, Korrektur- und Ergebnisoberfläche.	Formulare, Fotoübergabe, API-Aufruf, Nutzerkorrektur und Ergebnisdarstellung.	App-Framework und UI-Komponenten.
Öffentliche Nährwertdaten	Referenzwerte für Lebensmittel und Rezepte.	Datenmodell, Zuordnung, Einheiten, Plausibilisierung und Verwendung im Feature Engineering.	Ursprüngliche Datenbestände. Konkrete Quellen und Lizenzen müssen vor Abgabe genannt werden.
KI-Unterstützung beim Programmieren	Unterstützung bei Boilerplate, Debugging, Refactoring und Formulierungen.	Architektur- und Fachentscheidungen, Prüfung der Vorschläge und Verantwortung für den finalen Code.	Generierte Vorschläge und sprachliche Unterstützung.

3.3 Unterstützungsnachweis vor der Abgabe

Unterstützung muss konkret benannt werden. Allgemeine Angaben wie „ETH Frau xxx“ oder „Claude oder anderes Tool“ reichen für eine finale Abgabe nicht aus. Für jede Unterstützung sollen Name, Organisation oder Tool, Art der Unterstützung, betroffener Projektteil und Umfang dokumentiert werden.

Person / Institution / Tool	Art der Unterstützung	Betroffener Projektteil	Status
ETH-Coaching: Frau Balmer	Fachliches Feedback, Coaching oder Bereitstellung von Infrastruktur – konkret beschreiben.	Generelle Einschätzung und Gewichtung zum Thema «eigenen KI»	Erfolgt
Dokumentationen zu Visual Studio, Claude Code, RoboFlow, Flutter	Boilerplate, Fehlersuche, Refactoring und Formulierungshilfe	Code und Dokumentation	Dokumentiert
Roboflow / Meta SAM2	Bereitgestellte Bildanalyse-, Segmentierungs- oder Hosting-Technologie.	Bildanalyse, Workflow	Dokumentiert; Version ergänzen

4. Funktions- und Schnittstellenübersicht

ExplainEat besteht aus einem Offline-Teil für Datenaufbau und Training sowie einem Laufzeit-Teil für die Bewertung einer konkreten Mahlzeit. Diese Trennung verhindert, dass Training und produktive Verarbeitung vermischt werden. Sie macht ausserdem sichtbar, an welcher Stelle externe Modelle verwendet werden und wo mein eigenes PyTorch-Modell arbeitet. In der Folge wird ExplainEat Schritt-für-Schritt erklärt.

ExplainEat – Funktions- und Schnittstellenübersicht

Prototyp / Proof of Concept

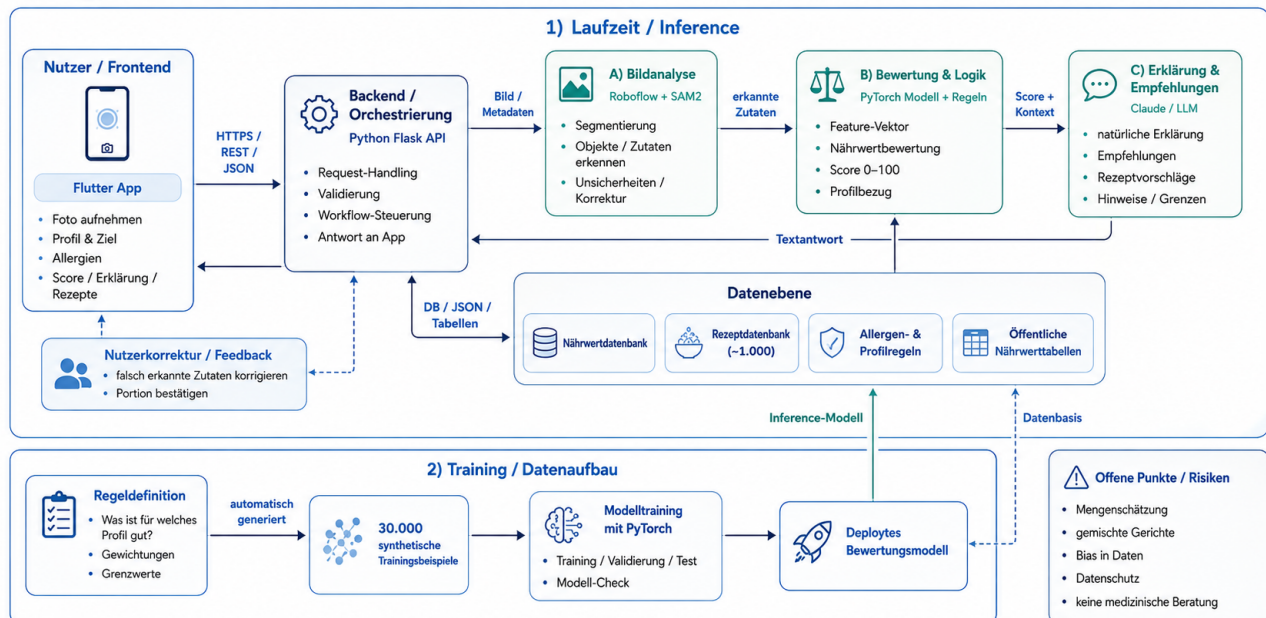


Abbildung 1: Funktions- und Schnittstellenübersicht von ExplainEat. Das Frontend ist funktional; der technische Schwerpunkt liegt auf Datenaufbau, Modelltraining, Inferenz und Integration.

Wichtig für die Ergebnisqualität: Ein Fehler in einem frühen Schritt – zum Beispiel eine falsche Zutat oder Portion – kann alle folgenden Berechnungen beeinflussen. Deshalb enthält der Ablauf einen verpflichtenden Korrekturschritt durch den Nutzer und eine separate Plausibilitätsprüfung vor der Erklärung.

4.1 Schritt 1: Profil und Ziel erfassen

- Ich erfasse nur Werte, die für die Bewertung benötigt werden, zum Beispiel Ziel, Aktivitätsniveau, Gewicht, Allergien oder Ernährungspräferenzen.
- Ich prüfe Wertebereiche auf Plausibilität und verhindere offensichtlich unrealistische Eingaben.
- Medizinisch sensible Angaben werden nicht wie gewöhnliche Fitnessziele behandelt.
- Kategorien werden strukturiert gespeichert, damit sie später eindeutig codiert werden können.

4.2 Schritt 2: Mahlzeit aufnehmen und ergänzen

- Der Nutzer nimmt ein Foto auf oder lädt ein Bild hoch.
- Zusätzliche Angaben wie Mahlzeitentyp, Sauce, Öl, Zubereitung, Verpackungswerte oder Portion können ergänzt werden.
- Die App weist darauf hin, dass ein Foto Salz, Zucker, versteckte Zutaten und Portionsgrösse nicht zuverlässig zeigt.
- Bild und strukturierte Metadaten werden gemeinsam an das Backend übertragen.

4.3 Schritt 3: Anfrage im Flask-Backend orchestrieren

- Das Backend nimmt Bild- und JSON-Daten über eine REST-Schnittstelle entgegen.
- Dateiformat, Pflichtfelder, Wertebereiche und technische Fehler werden validiert.
- Das Backend steuert die Reihenfolge von Bildanalyse, Nutzerbestätigung, Datenanreicherung, Modellaufruf und Erklärung.
- API-Schlüssel bleiben im Backend und werden nicht in der App gespeichert.
- Alle Teilergebnisse werden in einem kontrollierten Ergebnisobjekt zusammengeführt.

4.4 Schritt 4: Bildbereiche segmentieren und Lebensmittel erkennen

- SAM2 ist ein Segmentierungsmodell: Es kann Bildbereiche voneinander trennen, benennt Lebensmittel aber nicht automatisch zuverlässig.
- Für die Benennung ist zusätzlich eine Erkennungs- oder Klassifikationskomponente erforderlich. Die Dokumentation muss genau angeben, welches Modell dafür eingesetzt wird.
- Zu jedem Vorschlag werden Konfidenz und Modellversion gespeichert.
- Unsichere oder unbekannte Bestandteile werden sichtbar markiert und nicht als sichere Wahrheit behandelt.

4.5 Schritt 5: Nutzerbestätigung als Qualitätskontrolle

- Erkannte Zutaten werden vor der Bewertung angezeigt.
- Falsche Erkennungen können korrigiert, fehlende Zutaten ergänzt und Portionen bestätigt werden.
- Bestätigte Angaben haben Vorrang vor automatischen Schätzungen.
- Das Ergebnisobjekt unterscheidet gemessene, bestätigte und geschätzte Werte.

4.6 Schritt 6: Nährwerte und Kontext anreichern

- Bestätigte Zutaten werden Nährwertdaten zugeordnet.
- Einheiten werden vereinheitlicht und auf die bestätigte Portion umgerechnet.
- Zusätzliche Merkmale wie Gemüseanteil, Datenabdeckung oder Anteil unbekannter Zutaten werden berechnet.
- Fehlende Werte werden als fehlend markiert und nicht stillschweigend als Null interpretiert.

4.7 Schritt 7: Feature-Vektor erzeugen

- Alle Merkmale werden in eine feste, dokumentierte Reihenfolge gebracht.

- Kategorien werden mit Flags oder One-Hot-Encoding in Zahlen übersetzt.
- Numerische Werte werden mit Parametern skaliert, die ausschliesslich aus den Trainingsdaten berechnet wurden.
- Der resultierende Tensor hat den Datentyp float32 und die Form [Batchgrösse, Anzahl Merkmale].
- Training und Inferenz müssen exakt dasselbe Preprocessing verwenden.

4.8 Schritt 8: Score mit dem PyTorch-Modell berechnen

- Das Backend lädt Modellgewichte, Feature-Schema und Preprocessing-Artefakte gemeinsam.
- Das Modell wird in den Evaluationsmodus gesetzt und ohne Gradientenberechnung ausgeführt.
- Die Ausgabe wird auf erlaubte Wertebereiche begrenzt und auf technische Fehler geprüft.
- Allergien, harte Ausschlussregeln und medizinische Sicherheitsregeln bleiben ausserhalb des neuronalen Netzes.

4.9 Schritt 9: Plausibilität und Sicherheit prüfen

- Das System prüft, ob Portion, Zutaten und Datenabdeckung für eine belastbare Aussage ausreichen.
- Bei hoher Unsicherheit wird keine definitive Bewertung ausgegeben.
- Score, Warnhinweise und harte Regeln bleiben getrennte Ergebnisbestandteile.
- Die App muss ausdrücklich sagen können: „Eine zuverlässige Bewertung ist mit diesen Daten nicht möglich.“

4.10 Schritt 10: Erklärung mit LLM (Claude Code) erzeugen

- Claude erhält ein strukturiertes, bereits geprüftes Ergebnisobjekt und keine Verantwortung für die eigentliche Bewertung.
- Das Sprachmodell formuliert Score, Teilwerte, Stärken, offene Daten und Grenzen in verständliche Sprache um.
- Die Ausgabe wird auf Orientierung, nachvollziehbare Verbesserungen und passende Rezeptvorschläge begrenzt.
- Diagnosen, Therapie-, Medikamenten- oder Dosierungsempfehlungen sind ausgeschlossen.

4.11 Schritt 11: Ergebnis in der App anzeigen

- Die App zeigt bestätigte Zutaten, Datenqualität, Score, Teilbewertungen, Erklärung und mögliche Alternativen.
- Unsicherheit wird sichtbar dargestellt, damit keine Scheingenauigkeit entsteht.
- Der Prototypstatus und die fehlende medizinische Validierung werden klar gekennzeichnet.
- Der Nutzer kann Angaben korrigieren und die Bewertung erneut ausführen.

4.12 Schritt 12: Feedback kontrolliert weiterverwenden

- Korrekturen können als mögliche neue Datenpunkte gespeichert werden.
- Feedback wird nicht ungeprüft in das Training übernommen.
- Neue Daten werden bereinigt, versioniert und gegen feste Testfälle geprüft.
- Eine neue Modellversion wird nur übernommen, wenn die Verbesserung mit definierten Metriken messbar ist.

5. Der eigene KI-Kern: PyTorch-Modell und Feature-Vektor

5.1 Warum PyTorch im Mittelpunkt steht

PyTorch ist das Framework, mit dem ich das neuronale Netz definiere, trainiere, speichere und ausführe. Tensoren stellen numerische Daten dar, ein DataLoader liefert Trainingsbeispiele in Batches, das nn.Module

beschreibt das Netz, Autograd berechnet die Gradienten und ein Optimizer passt die Gewichte an. Meine Eigenleistung liegt nicht in diesen Standardbausteinen, sondern in ihrer Anbindung und fachlich begründeten Kombination für ExplainEat.

- Für die Challenge zeigt PyTorch einen vollständigen Machine-Learning-Prozess und nicht nur den Aufruf einer fertigen KI-API.
- Für das Lernen kann ich Vorwärtslauf, Fehlerfunktion, Backpropagation, Optimierung und Inferenz praktisch erklären.
- Für den Ausbau kann ich reale Daten, zusätzliche Ausgaben und Unsicherheitsmodelle ergänzen.
- Der Nachteil ist die geringere Transparenz gegenüber einer direkten Regelberechnung. Deshalb sind Baseline-Vergleich, Tests und Versionierung notwendig.

5.2 Lernaufgabe und Beziehung zur Regel-Engine

Die aktuellen Zielwerte werden aus einer von mir definierten Regel-Engine erzeugt. Das neuronale Netz lernt somit zunächst, diese Regeln aus dem Feature-Vektor möglichst gut nachzubilden. Technisch ist das ein Surrogat- oder Approximationsmodell: Es ersetzt die Regel-Engine nicht automatisch durch „bessere Intelligenz“, sondern versucht, deren Ergebnis zu reproduzieren.

Kritische Einordnung: Ein sehr gutes Ergebnis auf synthetischen Daten beweist vor allem, dass das Modell die selbst erzeugten Regeln gelernt hat. Es beweist noch nicht, dass die Bewertung auf echten Mahlzeiten ernährungswissenschaftlich korrekt oder für neue Bevölkerungsgruppen verlässlich ist.

Der fachliche Mehrwert des Modells muss deshalb gegen eine direkte Regelberechnung geprüft werden. Mögliche Vorteile wären eine glattere Bewertung, bessere Generalisierung bei leicht veränderten Eingaben oder die spätere Nutzung fachlich bewerteter realer Daten. Falls kein messbarer Vorteil entsteht, ist die Regel-Engine die transparentere Baseline und das PyTorch-Modell bleibt ein Lern- und Integrationsnachweis.

5.3 Feature-Vektor: verständliche Definition

Der Feature-Vektor ist eine geordnete Liste von Zahlen, die eine Mahlzeit und den relevanten Kontext für das Modell beschreibt. Jede Position hat eine feste Bedeutung. Beispiel: Position 1 kann die Energie, Position 2 das Protein und eine spätere Position das Aktivitätsniveau enthalten. Die Reihenfolge darf zwischen Training und Inferenz nie abweichen.

- Numerische Merkmale: zum Beispiel Energie, Protein, Kohlenhydrate, Fett, Ballaststoffe, Zucker, Salz, Portion und Gemüseanteil.
- Kategorische Merkmale: zum Beispiel Mahlzeitentyp oder Ziel; sie werden mit Flags oder One-Hot-Encoding in Zahlen übersetzt.
- Qualitätsmerkmale: Datenabdeckung, Anteil unbekannter Zutaten, Konfidenz oder Kennzeichnung einer Nutzerbestätigung.
- Sicherheitsmerkmale: Allergien und harte medizinische Ausschlüsse sollten nicht nur im Modell liegen, sondern separat regelbasiert geprüft werden.

5.4 Technische Bedeutung des aktuellen KI-Kerns

Der aktuelle KI-Kern zeigt, dass ich einen vollständigen ML-Lebenszyklus technisch umsetzen kann. Seine wissenschaftliche Aussagekraft ist jedoch durch synthetische Zielwerte begrenzt. Der nächste qualitative Schritt besteht nicht in einem grösseren Netz, sondern in besseren, realen und fachlich geprüften Daten sowie in einem klaren Vergleich mit einfachen Baselines. Siehe kritische Reflexion.

6. Training, Evaluation und Ausbau des Modells

6.1 Datengenerierung

Aus den definierten Regeln erzeuge ich rund 30'000 synthetische Kombinationen aus Mahlzeitenmerkmalen, Profilm Merkmalen und Zielscore. Der Vorteil ist, dass ich viele kontrollierte Fälle erzeugen und seltene Kombinationen gezielt abdecken kann. Der Nachteil ist, dass die Daten nur die Annahmen und Grenzen meiner Regeln enthalten. Fehler oder Verzerrungen im Regelwerk werden dadurch ebenfalls gelernt.

- Jeder Datensatz muss eine Versionsnummer und einen dokumentierten Generatorstand besitzen.
- Zulässige Wertebereiche und Verteilungen müssen beschrieben werden.
- Unrealistische Kombinationen sollen vermieden oder ausdrücklich als Testfälle markiert werden.
- Trainingsdaten dürfen keine Informationen aus Validierungs- oder Testdaten übernehmen.

6.2 Datenaufteilung und Preprocessing

1. Die Daten werden reproduzierbar in Training, Validierung und Test aufgeteilt.
2. Der Testanteil bleibt bis zur finalen Bewertung unberührt.
3. Mittelwerte, Standardabweichungen oder andere Skalierungsparameter werden ausschliesslich auf dem Trainingsanteil berechnet.
4. Kategorien und Feature-Reihenfolge werden in einem versionierten Schema gespeichert.
5. Das Backend verwendet exakt dieselben Transformationen wie das Trainingsskript.

6.3 Trainingsablauf

1. Regeln, Feature-Schema und Zieldefinition festlegen.
2. Synthetische Trainingsbeispiele erzeugen und auf Plausibilität prüfen.
3. Trainings-, Validierungs- und Testdaten reproduzierbar trennen.
4. Modell trainieren und den Validierungsfehler beobachten.
5. Besten Modellzustand speichern; nicht nur den letzten Trainingsstand.
6. Modell, Feature-Schema, Skalierung, Konfiguration und Metriken gemeinsam versionieren.
7. Finale Bewertung auf dem unabhängigen Testanteil durchführen.
8. Ergebnis mit der direkten Regel-Engine und einfachen Baselines vergleichen.

6.4 Ausbaupfad der KI

- Stufe 1 – Reproduzierbarkeit: Seeds, Versionen, Feature-Schema, Split, Modellhash und Beispielinputs fixieren.
- Stufe 2 – Reale Testbilder: kleiner, manuell geprüfter Bilddatensatz mit einfachen und gemischten Mahlzeiten.
- Stufe 3 – Bestätigte Portionen: automatische Schätzungen durch Nutzerbestätigung ergänzen und Restunsicherheit dokumentieren.
- Stufe 4 – Fachlich bewertete Labels: Zielwerte durch qualifizierte Fachpersonen prüfen lassen, statt sie nur aus eigenen Regeln abzuleiten.
- Stufe 5 – Multi-Output-Modell: Gesamtscore, Teilwerte und Unsicherheit getrennt ausgeben.
- Stufe 6 – Active Learning: Fälle priorisieren, bei denen Modell, Regel-Engine und Nutzerkorrektur stark abweichen.
- Stufe 7 – Bias- und Subgruppenprüfung: Esskulturen, Gerichtstypen, Länder, Profile und Datenqualitätsstufen getrennt testen.
- Stufe 8 – Monitoring: Datensatz-, Feature-, Modell- und Metrikversionen mit bekannten Grenzen verknüpfen.

7. Zwei Use Cases und ihr Nutzen

7.1 Use Case 1 – gesunder Kraft- und Fitnesssportler

Der erste Use Case zeigt den vorgesehenen Kernnutzen in einem nicht-medizinischen Szenario. Ein gesunder Erwachsener möchte seine Mahlzeit im Zusammenhang mit Krafttraining und allgemeiner Fitness verstehen.

Aspekt	Beispiel
Profil	27 Jahre, gesund, vier Trainingseinheiten pro Woche, Ziel Muskelaufbau und allgemeine Fitness.
Mahlzeit	Poulet, Reis, Brokkoli und Sauce.
Bildanalyse	Lebensmittel werden als Vorschläge erkannt; die Sauce erhält eine geringere Konfidenz.
Nutzerkorrektur	Portionen und Sauce werden manuell bestätigt.
Feature-Vektor	Zum Beispiel Energie, Protein, Kohlenhydrate, Gemüseanteil, Portion, Aktivität, Ziel und Datenqualität.
PyTorch-Ausgabe	Profilbezogener Gesamtscore und – falls implementiert – Teilwerte.
Claude-Ausgabe	Verständliche Erklärung, warum bestätigte Proteinquelle und Gemüseanteil passen und wo Unsicherheit bleibt.

Nutzen dieses Profils

- Die Bildanalyse erzeugt eine erste Zutatenliste und reduziert den manuellen Aufwand.
- Die Bewertung berücksichtigt nicht nur Kalorien, sondern auch Ziel, Portion und Datenqualität.
- Teilwerte und Gründe sind verständlicher als eine einzelne Zahl ohne Kontext.
- Der Nutzer erkennt wiederkehrende Muster und kann Änderungen vergleichen.
- Der Score garantiert weder Muskelaufbau noch einen individuellen Leistungsfortschritt.

Beispielausgabe: Die Mahlzeit passt grundsätzlich gut zum hinterlegten Trainingsziel. Die bestätigte Proteinquelle und der Gemüseanteil sprechen für eine ausgewogene Hauptmahlzeit. Die Sauce konnte nur teilweise bewertet werden; deshalb ist die Datenqualität mittel.

7.2 Use Case 2 – medizinisch sensibles Profil

Dieser Use Case gehört nicht zum medizinisch validierten Kern des Prototyps. Er zeigt lediglich, wie ExplainEat in einer späteren, fachlich geprüften Ausbaustufe mehrere bestätigte Datenpunkte transparent zusammenführen könnte. Individuelle Grenzwerte dürfen nicht von der App erfunden werden.

Aspekt	Beispiel
Profil	58 Jahre, Diabetes und erhöhter Blutdruck. Zielbereiche stammen ausschliesslich von einer qualifizierten Fachperson.
Mahlzeit	Fertigsuppe, Weissbrot, Wurst und gesüsstes Getränk.
Bildanalyse	Gerichtsklassen werden vorgeschlagen; genaue Produktwerte sind aus dem Foto nicht zuverlässig ableitbar.
Zusatzdaten	Etikett- oder Barcode-Daten ergänzen Kohlenhydrate, Zucker, Salz/Natrium und Portion.
Regelprüfung	Datenabdeckung, Portion, verarbeitete Produkte und hinterlegte fachliche Grenzen werden geprüft.
PyTorch-Rolle	Nur ein fachlich validiertes Profilmodell dürfte eine vorsichtige Passungsbewertung liefern.
Claude-Rolle	Neutral erklären, welche bestätigten Merkmale beitragen, und welche Daten fehlen; keine Therapie.

Nutzen dieses Ausbauprofils

- Produktetiketten und Bildinformationen können gemeinsam dargestellt werden.
- Die App kann erklären, warum eine Bewertung unsicher ist und welche Daten fehlen.
- Mehrere profilrelevante Merkmale werden in einer verständlichen Zusammenfassung verbunden.
- Eine sachliche Alternative aus der Rezeptdatenbank kann vorgeschlagen werden, ohne Lebensmittel pauschal zu verbieten.
- Das Ergebnis kann als Gesprächsgrundlage dienen, ersetzt aber keine fachliche Entscheidung.

Zwingende Sicherheitsgrenze: Keine Diagnose, keine Anpassung von Insulin oder Medikamenten, keine Garantie zur Blutzucker- oder Blutdruckwirkung und keine pauschale Freigabe „für Diabetiker geeignet“. Bei unvollständigen Daten muss ExplainEat die Bewertung begrenzen oder ablehnen.

7.3 Vergleich der Use Cases

Dimension	Gesunder Sportler	Medizinisch sensibles Ausbauprofil
Hauptziel	Orientierung für Fitness- und Trainingsziel.	Verständliche Zusammenführung bestätigter Profil- und Produktdaten.
Bedeutung des Fotos	Schneller Einstieg, danach Bestätigung.	Nur erster Hinweis; Etikett- und Fachwerte sind entscheidend.
Rolle des PyTorch-Modells	Profilbezogene Bewertung mit überschaubarem Risiko.	Nur nach fachlicher Validierung und strengeren Schutzregeln.
Rolle der Regeln	Allergien, Plausibilität, Datenqualität und Unsicherheit.	Zusätzlich medizinische Ausschlüsse und fachlich definierte Grenzwerte.
Rolle von Claude	Erklärung und allgemeine Vorschläge.	Vorsichtige Erklärung; keine Diagnose oder Therapie.
Hauptrisiko	Scheingenauigkeit und fehlender Tageskontext.	Gesundheitsschaden durch falsche oder zu definitive Aussagen.

8. Risiken, Datenschutz und fachliche Grenzen

8.1 Fehlerfortpflanzung

ExplainEat ist eine Verarbeitungskette. Ein Fehler bei Erkennung, Portion, Datenzuordnung oder Skalierung kann den Score und anschliessend auch die Erklärung verfälschen. Die Nutzerbestätigung und die Plausibilitätsprüfung sind deshalb keine Komfortfunktionen, sondern technische Kontrollpunkte.

Fehlerquelle	Mögliche Wirkung	Kontrolle
Falsche Lebensmittelerkennung	Falsche Nährwerte und falscher Score	Konfidenz anzeigen, Nutzerkorrektur verlangen, unbekannte Klasse zulassen
Unbekannte Portion	Scheingenauigkeit bei Energie und Nährstoffen	Portion bestätigen oder Ergebnis begrenzen
Fehlende / falsche Nährwertdaten	Systematische Verzerrung	Quelle, Einheit, Version und Datenabdeckung dokumentieren
Abweichendes Preprocessing	Training und Inferenz liefern unvereinbare Ergebnisse	Gemeinsames versioniertes Feature-Schema und Tests
LLM ergänzt Fakten	Plausibel formulierte, aber falsche Erklärung	Nur strukturiertes Ergebnisobjekt; Fakten dürfen nicht erfunden werden

8.2 Bias und Datenlücken

- Synthetische Daten spiegeln die Annahmen der Regel-Engine wider und können deren Verzerrungen verstärken.
- Lebensmittel aus unterschiedlichen Esskulturen können in Bild- und Nährwertdaten ungleich vertreten sein.
- Gemischte Gerichte, Saucen, Getränke und verarbeitete Produkte sind schwieriger zu erkennen als klar getrennte Lebensmittel.
- Ein Gesamtwert kann Unterschiede nach Ziel, Portion, Alter, Aktivität oder Datenqualität verdecken; deshalb sind Subgruppenmetriken nötig.
- Die Lösung darf keine bestimmten Körperformen, extremen Diäten oder übermäßige Einschränkungen fördern.

8.3 Datenschutz und technische Sicherheit

- Es werden nur Daten erhoben, die für die konkrete Bewertung erforderlich sind.
- API-Schlüssel werden nicht im Flutter-Client gespeichert.
- Bilder und Profilwerte sollten nicht länger gespeichert werden als technisch notwendig.
- Gesundheitsbezogene Angaben benötigen besonders klare Zweckbindung, Zugriffsschutz und Löschregeln.
- An externe Dienste werden nur die minimal notwendigen Daten übertragen; personenbezogene Details sollen entfernt oder reduziert werden.
- Logs dürfen keine geheimen Schlüssel oder unnötige Gesundheitsdaten enthalten.

8.4 Medizinische und fachliche Grenzen

ExplainEat bewertet Daten nach einem technischen Modell und einer selbst definierten Logik. Ohne fachliche Validierung darf die Anwendung keine medizinische Eignung behaupten. Das gilt besonders für Diabetes, Allergien, Blutdruck, Medikamente, Essstörungen oder andere gesundheitlich sensible Situationen.

- Keine Diagnose und keine Therapieentscheidung.
- Keine Medikamenten-, Insulin- oder Dosierungsempfehlung.
- Keine Garantie für körperliche Wirkung, Gewichtsveränderung oder sportlichen Erfolg.
- Keine definitive Bewertung bei fehlenden, geschätzten oder widersprüchlichen Daten.
- Harte Allergie- und Ausschlussregeln dürfen nicht allein von einem statistischen Modell abhängen.

9. Gesamtbeurteilung und offene Punkte vor der Abgabe

9.1 Gesamtbeurteilung

ExplainEat ist als Schülerprojekt technisch anspruchsvoll, weil mehrere Komponenten zu einem vollständigen Laufzeitprozess verbunden werden. Besonders positiv sind die klare Abgrenzung externer KI-Bausteine, die Nutzerkorrektur als Qualitätskontrolle und die kritische Begrenzung medizinischer Aussagen.

Die Dokumentation ist erst dann vollständig belastbar, wenn zentrale Angaben aus dem tatsächlichen Code übernommen und durch reproduzierbare Tests belegt sind. Dazu gehören insbesondere die genaue Modellarchitektur, der vollständige Feature-Vektor, die verwendete Erkennungskomponente neben SAM2, die exakte Metrik hinter „98 Prozent“, konkrete Datenquellen und der Unterstützungsnachweis.

9.2 Priorisierte Prüfpunkte vor der Abgabe

Priorität	Prüfpunkt	Dringlichkeit
1	Repository auf einem fremden Computer starten oder vollständiges Screenrecording erstellen.	Kritisch

Priorität	Prüfpunkt	Dringlichkeit
2	Exakte Modellarchitektur und Hyperparameter aus dem Code übernehmen.	Erledigt — Anhang A und B
3	Verbindliche Feature-Reihenfolge, Einheiten und Skalierung ergänzen.	Erledigt — Anhang A
4	98-Prozent-Aussage mit Metrik, Toleranz, Testgrösse und Rechenweg belegen.	Erledigt — Anhang B.1
5	Regel-Engine und einfache Baselines mit dem Modell vergleichen.	Erledigt — <code>scripts/evaluate_model.py</code>
6	Klar benennen, welches Modell Lebensmittel erkennt; SAM2 allein ist Segmentierung.	Hoch
7	Nährwert- und Rezeptdaten mit Quelle und Lizenz dokumentieren.	Erledigt — Abschnitt 10, [11] und [12]
8	Alle Personen, Tools und Arten der Unterstützung konkret nennen.	Hoch
9	API-Schlüssel, Datenschutz, Logging und Fehlerfälle im README dokumentieren.	Mittel
10	Video-Pitch auf maximal zwei Minuten kürzen und die grösste Grenze ausdrücklich nennen.	Mittel

Empfehlung aus Auditor-Sicht: Die Dokumentation soll nicht den Eindruck erwecken, dass jeder beschriebene Soll-Schritt bereits vollständig implementiert und getestet ist. Eine kurze, ehrliche Kennzeichnung „implementiert“, „teilweise implementiert“ oder „geplant“ erhöht die Glaubwürdigkeit deutlich.

10. Quellen und technische Referenzen

Die folgende Liste enthält die offiziellen Referenzen, auf denen die technische Einordnung dieser Dokumentation basiert, sowie die Herkunft der tatsächlich verwendeten Nährwert- und Rezeptdaten (Einträge [11] und [12]).

Nr.	Quelle	Fundstelle
[1]	Schweizer KI Challenge – Teilnahmeinformationen und Bewertungskriterien	https://www.ki-challenge.ch/infos
[2]	Schweizer KI Challenge – Projektabgabe	https://www.ki-challenge.ch/abgabe
[3]	PyTorch – offizielle Dokumentation	https://docs.pytorch.org/docs/stable/
[4]	PyTorch – torch.nn.Module	https://docs.pytorch.org/docs/stable/generated/torch.nn.Module
[5]	PyTorch – DataLoader	https://docs.pytorch.org/docs/stable/data.html
[6]	Meta – Segment Anything Model 2 (SAM2)	https://github.com/facebookresearch/sam2
[7]	Roboflow – Object Detection / Inference	https://docs.roboflow.com/deploy/serverless/object-detection
[8]	Flask – offizielle Dokumentation	https://flask.palletsprojects.com/en/stable/
[9]	Flutter – offizielle Dokumentation	https://docs.flutter.dev/

Nr.	Quelle	Fundstelle
[10]	Anthropic – Claude Dokumentation	https://docs.anthropic.com/en/docs/welcome
[11]	Nährwertdaten	37 Grundlebensmittel, als typische Referenzwerte pro 100 g fest im Code hinterlegt (<code>explain_eat/nutrition.py</code>). Die Werte entsprechen gängigen öffentlichen Nährwerttabellen; eine einzelne Quelle je Wert ist nicht dokumentiert. Diese fehlende Rückverfolgbarkeit ist eine bekannte Grenze des Prototyps.
[12]	Rezeptdaten	1 000 Rezepte in <code>explain_eat/recipes.json</code> , programmatisch erzeugt mit <code>scripts/generate_recipes.py</code> aus üblichen Zutatenkombinationen. Keine externe Rezeptdatenbank und keine fremde Lizenz; die Zusammenstellung stammt aus dem Generator dieses Projekts.

Anhang A. Prüfschema für den Feature-Vektor

Dieser Anhang gibt den tatsächlich implementierten Feature-Vektor wieder, abgeglichen mit `explain_eat/ai_model.py`. Entscheidend ist die exakte Reihenfolge: Training und Inferenz verwenden dieselbe Funktion `encode_features()`, wodurch ein Auseinanderlaufen ausgeschlossen ist.

Die folgende Tabelle ist direkt aus `explain_eat/ai_model.py`, Funktion `encode_features()`, übernommen. Sie beschreibt den **tatsächlich implementierten** Stand (12 Merkmale) und nicht einen Zielzustand. Training und Inferenz verwenden dieselbe Funktion, wodurch abweichendes Preprocessing ausgeschlossen ist.

Index	Merkmal	Typ	Rohwert	Skalierung im Code
0	age	numerisch	Alter in Jahren	/ 100.0
1	weight	numerisch	Körpergewicht in kg	/ 150.0
2	activity	ordinal	Aktivitätsniveau	low = 0.0, moderate = 0.5, high = 1.0
3	goal_health	One-Hot	Ziel „health“	0/1, keine Skalierung
4	goal_muscle	One-Hot	Ziel „muscle“	0/1, keine Skalierung
5	goal_weight_loss	One-Hot	Ziel „weight_loss“	0/1, keine Skalierung
6	calories	numerisch	kcal der Mahlzeit	/ 1000.0
7	protein_g	numerisch	g Protein	/ 100.0
8	fat_g	numerisch	g Fett	/ 100.0
9	carbs_g	numerisch	g Kohlenhydrate	/ 150.0
10	fiber_g	numerisch	g Ballaststoffe	/ 50.0
11	sugar_g	numerisch	g Zucker	/ 100.0
separat	Allergien / harte Ausschlüsse	Regelwerk	—	nicht Teil des Feature-Vektors; separat regelbasiert geprüft

Ausgabeseite (9 Werte): Position 0 ergibt nach sigmoid den Score (0–1, in der App auf 0–100 skaliert). Die Positionen 1–4 sind die Flags `protein_low`, `fiber_low`, `sugar_high`, `calorie_mismatch`. Die Positionen 5–8 sind die Logits der Empfehlungsklassen `add_protein`, `add_vegetables`, `add_complex_carbs`, `balanced`.

Architektur: `Linear(12 → 64) → ReLU → Linear(64 → 64) → ReLU → Linear(64 → 9)`
(mehrschichtiges Perzeptron mit gemeinsamem Rumpf und drei Ausgabeköpfen).

Offene Punkte, ehrlich benannt: Merkmale wie Portionsgrösse, Gemüseanteil, Datenabdeckung, Anteil unbekannter Zutaten, Salz und Mahlzeitentyp sind in Abschnitt 5.3 als sinnvoll beschrieben, aber im aktuellen Stand **noch nicht** im Feature-Vektor enthalten. Sie gehören zum Ausbaupfad, nicht zum implementierten Prototyp.

A.1 Beispiel für die Transformation

Rohwert „Mahlzeitentyp = Abendessen“ wird nicht als Text an das Modell übergeben. Stattdessen wird für jede zulässige Kategorie ein Zahlenfeld verwendet. Bei vier Kategorien könnte die Codierung beispielsweise [Frühstück=0, Mittagessen=0, Abendessen=1, Snack=0] lauten. Die tatsächlichen Kategorien müssen aus dem Code übernommen werden.

Ein numerischer Rohwert wie Protein in Gramm wird mit den im Training berechneten Parametern skaliert. Dieselben Parameter müssen im Backend verwendet werden. Eine neu berechnete oder abweichende Skalierung in der App würde die Bedeutung der Modellgewichte verändern und zu falschen Resultaten führen.

Anhang B. Verbindliche Modell- und Reproduzierbarkeitsdaten

Alle Werte stammen aus einem reproduzierbaren Lauf. Nachvollziehbar mit:

```
python scripts/train_ai_model.py --samples 30000 --epochs 40 --seed 42
python scripts/evaluate_model.py --samples 30000 --seed 42
```

Nachweis	Finaler Wert
Repository	https://github.com/nik2012slapshot-lgtm/ExplainEat
Git-Commit	0867caf
Python-Version	3.13.14
PyTorch-Version	2.13.0+cpu
Datengenerator	<code>scripts/train_ai_model.py</code> , Funktion <code>build_dataset()</code>
Anzahl Beispiele	30 000 (synthetisch)
Train / Validation / Test	21 000 / 4 500 / 4 500 (70 / 15 / 15 %)
Random Seed	42 (<code>random.seed</code> und <code>torch.manual_seed</code>)
Anzahl Features	12 (siehe Anhang A)
Modellarchitektur	<code>Linear(12→64) → ReLU → Linear(64→64) → ReLU → Linear(64→9)</code>
Loss-Funktion	MSE (Score) + BCEWithLogits (Flags) + CrossEntropy (Empfehlung), summiert
Optimizer / Lernrate	Adam, <code>lr = 1e-3</code>
Batchgrösse / Epochen	256 / 40
Frühes Stoppen / bestes Modell	nicht implementiert — gespeichert wird der letzte Epochenstand
MAE (Score, Test)	7,96 Punkte auf der Skala 0–100

Nachweis	Finaler Wert
RMSE (Score, Test)	9,90
R ² (Score, Test)	0,758
Toleranzquote ± 5 Punkte	37,8 %
Maximalfehler / 95. Perzentil	35,71 / 19,06 Punkte
Empfehlungsklasse (Accuracy)	98,4 % (4 Klassen)
Flag-Genauigkeit	97,5 % (4 binäre Flags)
Baseline: Regel-Engine	MAE 0,00 — erzeugt die Zielwerte selbst, daher exakt
Baseline: lineare Regression	MAE 10,26 / R ² 0,599
Baseline: Konstante (Mittelwert)	MAE 16,49 / R ² -0,000
Modellartefakt	explain_eat/models/nutrition_ai.pt, SHA-256 (16): c996262f21b7bb16
Feature-Schema	fest im Code: <code>encode_features()</code> in <code>explain_eat/ai_model.py</code>
Preprocessing-Artefakt	keines — die Skalierung besteht aus festen Divisoren im Code (siehe Anhang A)

B.1 Einordnung der Messwerte

Wozu die oft genannte Zahl „98 Prozent“ gehört. Die 98,4 Prozent sind die Trefferquote der **Empfehlungsklasse** — also die Frage, welche von vier Kategorien (`add_protein`, `add_vegetables`, `add_complex_carbs`, `balanced`) vorgeschlagen wird. Sie sind **nicht** die Genauigkeit des Scores und **keine** Aussage über ernährungswissenschaftliche Richtigkeit.

Der Score ist deutlich ungenauer. Mit einem mittleren absoluten Fehler von rund 8 Punkten und einer Toleranzquote von nur 37,8 Prozent innerhalb von ± 5 Punkten ist der Score als grobe Einordnung brauchbar, aber nicht als präziser Messwert. In der App sollte er deshalb nicht mit Nachkommastellen oder als exakte Zahl dargestellt werden.

Das Modell schlägt die einfachen Baselines. Gegenüber der linearen Regression (MAE 10,26) verbessert das Netz den Fehler um rund 22 Prozent, gegenüber der konstanten Vorhersage (MAE 16,49) um mehr als die Hälfte. Die nichtlineare Architektur ist damit begründet und nicht bloss Selbstzweck.

Die entscheidende Einschränkung bleibt. Die Regel-Engine erreicht per Konstruktion einen Fehler von null, weil sie die Zielwerte selbst erzeugt. Das neuronale Netz ist deshalb ein **Surrogatmodell**: Es lernt, meine eigenen Regeln nachzubilden. Ein hoher Messwert belegt technische Umsetzung und Generalisierung innerhalb dieser Regeln — er belegt **nicht**, dass die Regeln ernährungswissenschaftlich korrekt sind. Der nächste inhaltliche Schritt sind fachlich geprüfte Zielwerte, nicht ein grösseres Netz.